

東京 2025 LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

Applying Program Verification to Linux Kernel Code: Challenges, Practices, and Automation

Keisuke Nishimura, Jean-Pierre Lozi, Julia Lawall



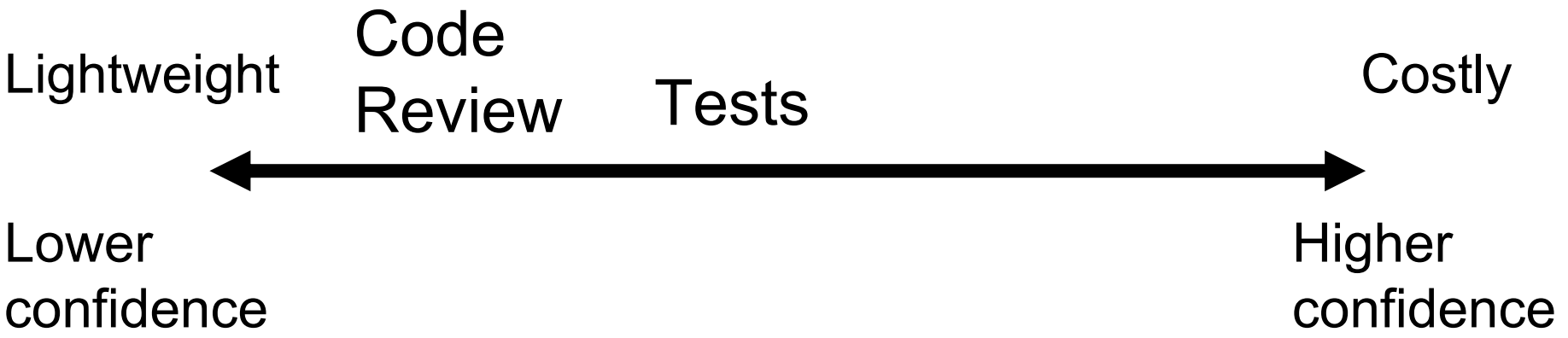
東京 2025 Linux Plumbers Conference
TOKYO, JAPAN / DEC. 11-13, 2025

The Linux Kernel

- In theory, the operating system should always run properly.
- In reality, a lot of bugs:
  - Known bug patterns (e.g., NULL dereference)
    - There are many tools to detect them (but there are still many such bugs!)
  - Semantic bugs (e.g., missing privilege checks)
    - These are hard to detect systematically.

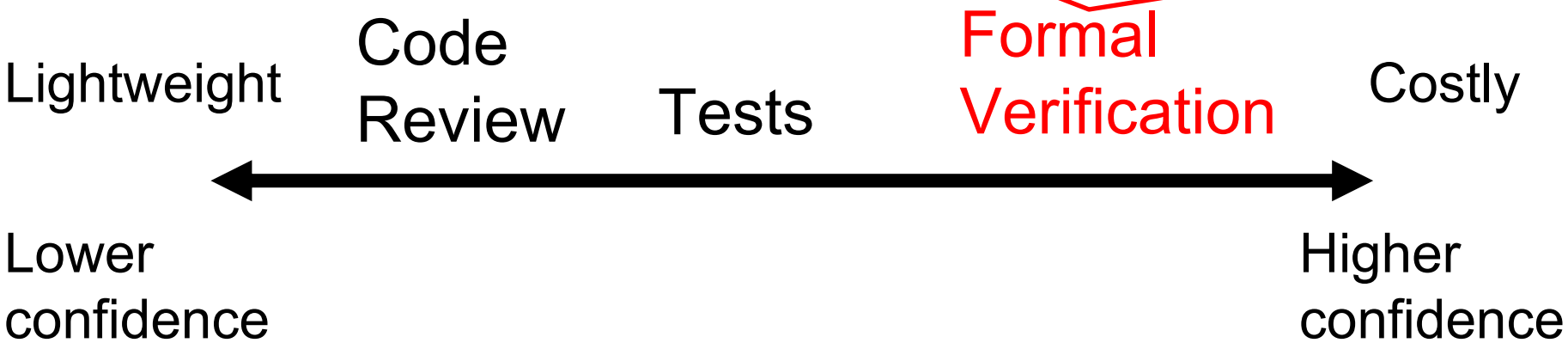
Work in Progress - License: CC-BY-4.0

Detecting Bugs



Detecting Bugs

Our message: Applying formal verification is hard, but it is becoming increasingly feasible.



Outline

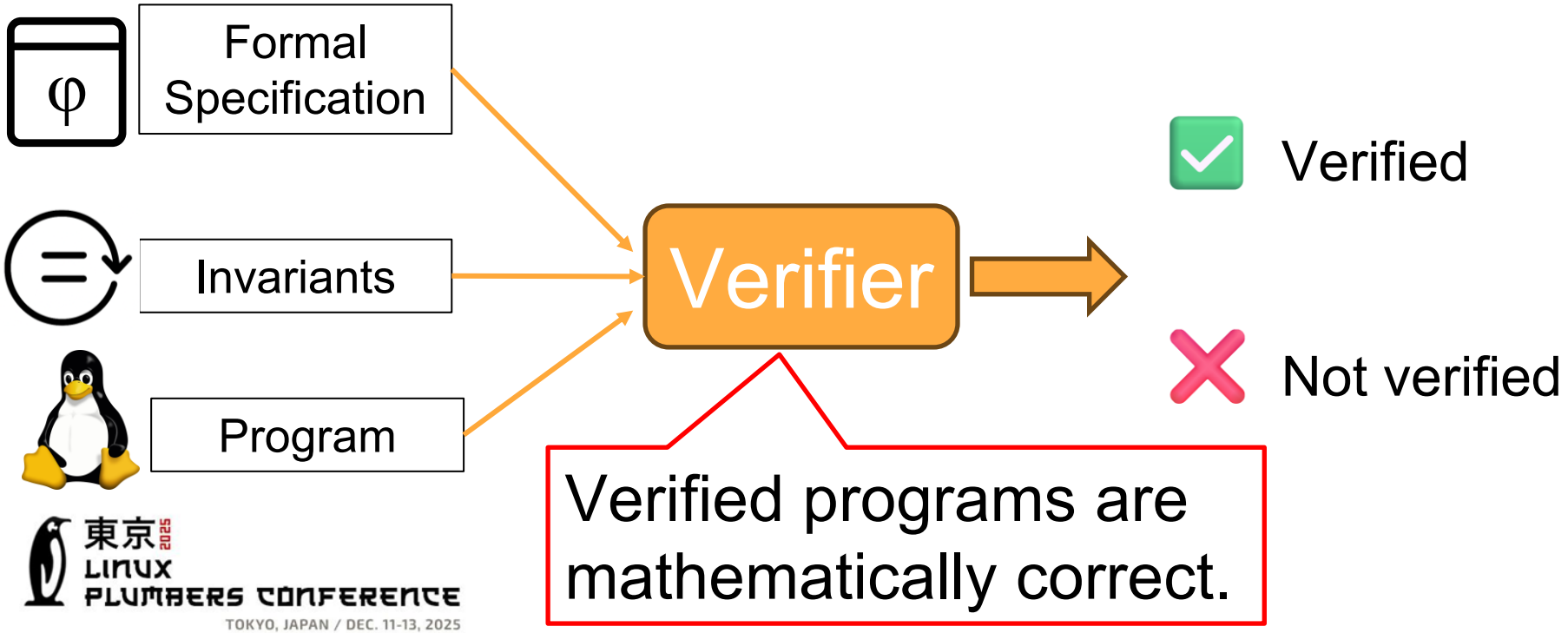
- Overview: What is program verification?
- Case Study: Applying verification to a small function, is_core_idle()
- Towards practical verification of the Linux kernel



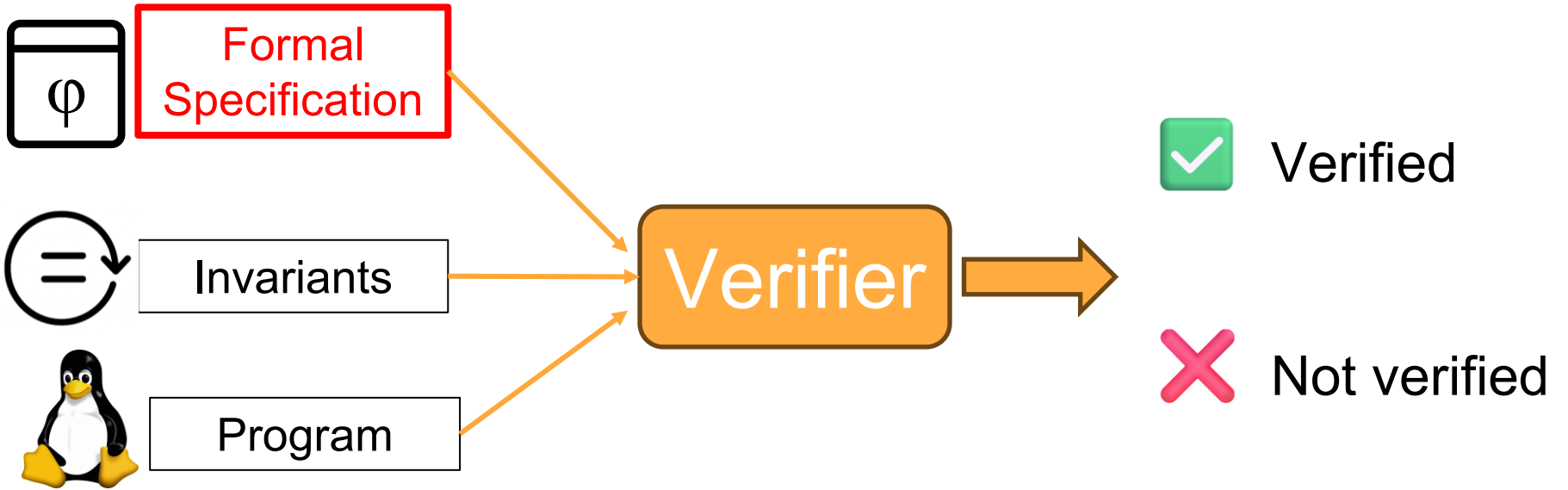
Outline

- Overview: What is program verification?
- Case Study: Applying verification to a small function, is_core_idle()
- Towards practical verification of the Linux kernel

Deductive Program Verification: Overview



Deductive Program Verification: Overview



Specifications in Deductive Program Verification

- The verifier decides if a given program is correct.
- A correct program Prog is defined using the Hoare triple:

$$\{ \text{Pre-condition} \}\ \text{Prog}\ \{ \text{Post-condition} \}$$

Specifications in Deductive Program Verification

- The verifier decides if a given program is correct.
- A correct program Prog is defined using the Hoare triple:

{ Pre-condition } Prog { Post-condition }

If this pre-condition holds.

Specifications in Deductive Program Verification

- The verifier decides if a given program is correct.
- A correct program Prog is defined using the Hoare triple:

$$\{ \text{Pre-condition} \}\ \text{Prog}\ \{ \text{Post-condition} \}$$

Then, after the execution of Prog

Specifications in Deductive Program Verification

- The verifier decides if a given program is correct.
- A correct program Prog is defined using the Hoare triple:

$$\{ \text{Pre-condition} \}\ \text{Prog}\ \{ \text{Post-condition} \}$$

This post-condition holds

How to Represent Specifications

- One possible approach: Natural Language

```
If the variable x is more than 1,
  after the execution of x:=x+1,
  the variable x is more than 2.
```

We use a “formal” representation of specifications.

Formal Specifications

- Formal verification needs “formal” specifications.

$$\{ x > 1 \}\ x := x+1\ \{ x > 2 \}$$

- Writing formal specifications:

  ☑ Good: No Ambiguity and Machine-checkable Correctness

  ✕ Bad: Significant Efforts

Work in Progress - License: CC-BY-4.0

Machine-checkable Correctness of Specifications

Typical Specification

- If a condition X holds, then the return value is 1.
- If a condition Y holds, then the return value is 2.
- If a condition Z holds, then the return value is 3.



Is this specification good?

- Are the conditions disjoint?
- Are the conditions complete?

Formal specifications allows for the systematic disjoint/complete-ness checks.

Specification Correctness

- In the specifications of __ftrace_event_enable_disable, one “break” may not be considered?
- This is triggered when a function, call->class->reg() returns an error
- A minor path in a big function might be overlooked.

```
case 1:
        /*
         * When soft_disable is set and enable is set, we want to
         * register the tracepoint for the event, but leave the event
         * as is. That means, if the event was already enabled, we do
         * nothing (but set soft_mode). If the event is disabled, we
         * set SOFT_DISABLED before enabling the event tracepoint, so
         * it still seems to be disabled.
         */
        if (!soft_disable)
                clear_bit(EVENT_FILE_FL_SOFT_DISABLED_BIT, &file->flags);
        else {
                if (atomic_inc_return(&file->sm_ref) > 1)
                        break;
                soft_mode = true;
                /* Enable use of trace_buffered_event */
                trace_buffered_event_enable();
        }

        if (!(file->flags & EVENT_FILE_FL_ENABLED)) {
                bool cmd = false, tgid = false;

                /* Keep the event disabled, when going to soft mode. */
                if (soft_disable)
                        set_bit(EVENT_FILE_FL_SOFT_DISABLED_BIT, &file->flags);

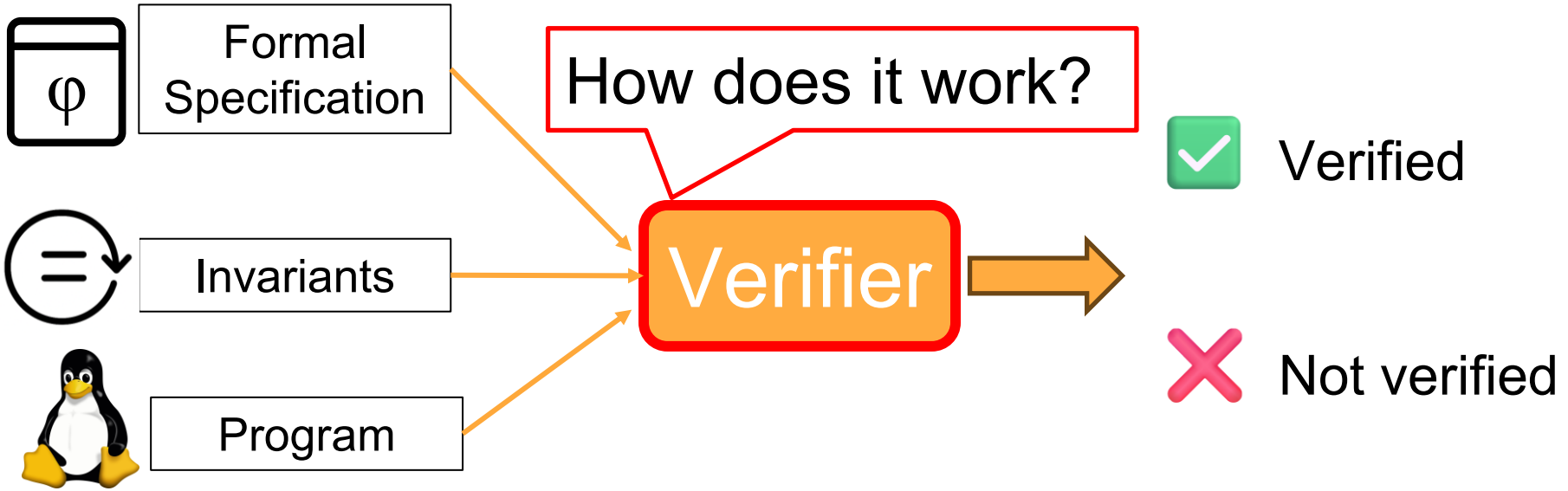
                if (tr->trace_flags & TRACE_ITER_RECORD_CMD) {
                        cmd = true;
                        tracing_start_cmdline_record();
                        set_bit(EVENT_FILE_FL_RECORDED_CMD_BIT, &file->flags);
                }

                if (tr->trace_flags & TRACE_ITER_RECORD_TGID) {
                        tgid = true;
                        tracing_start_tgid_record();
                        set_bit(EVENT_FILE_FL_RECORDED_TGID_BIT, &file->flags);
                }

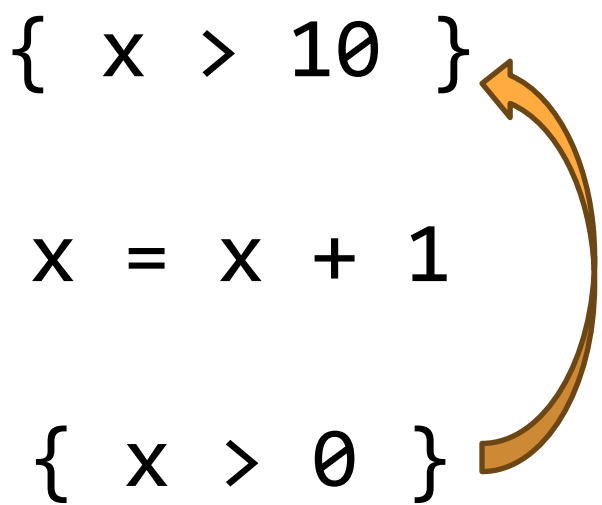
                ret = call->class->reg(call, TRACE_REG_REGISTER, file);
                if (ret) {
                        if (cmd)
                                tracing_stop_cmdline_record();
                        if (tgid)
                                tracing_stop_tgid_record();
                        pr_info("event trace: Could not enable event "
                                "%s\n", trace_event_name(call));
                        break;
                }
                set_bit(EVENT_FILE_FL_ENABLED_BIT, &file->flags);

                /* WAS_ENABLED gets set but never cleared. */
                set_bit(EVENT_FILE_FL_WAS_ENABLED_BIT, &file->flags);
```

Deductive Program Verification: Overview



The Internals of a Verifier



Finding the weakest preconditions by reasoning in the backward direction

The weakest preconditions: `x > -1`

Checking if the weakest preconditions implies the given precondition

`x > 10` ⇒ `x > -1`

Given Precondition — Weakest Precondition

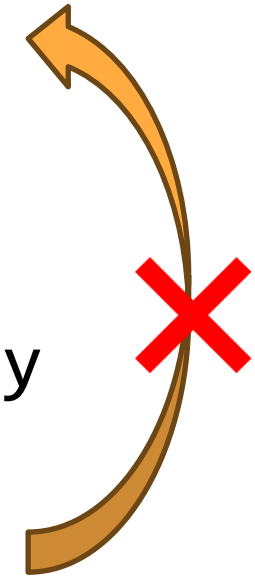
Problem: Verifying a Loop

```
{ ??? }

while(c)

    loop-body

{ x > 0 }
```

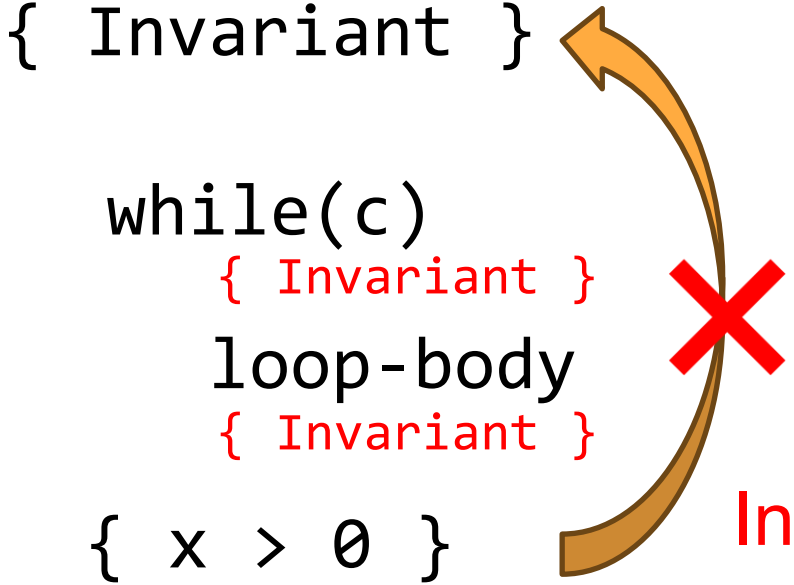


~~Backward reasoning~~

How many iterations?

Does it terminate?

Problem: Verifying a Loop



~~Backward reasoning~~

How many iterations?

Does it terminate?

Instead of automatic reasoning, we need to write invariants that hold before and after a loop.

Manual Annotation of Loop Invariants

- Writing invariants for every loop in a function to be verified can be a major stumbling block in verification.

- For practical verification of the Linux kernel, some degree of automation is needed and doable; we will see it later.

Outline

- Overview: What is program verification?
- Case Study: Applying verification to a small function, is_core_idle()
- Towards practical verification of the Linux kernel



Example: Verification of is_core_idle()

- is_core_idle() checks if all CPUs are idle.
  - This iterates over CPU IDs in a cpumask (a set of CPUs).
  - Once a non-idle CPU is found, it returns false immediately.
  - If all CPUs are idle, it returns true.

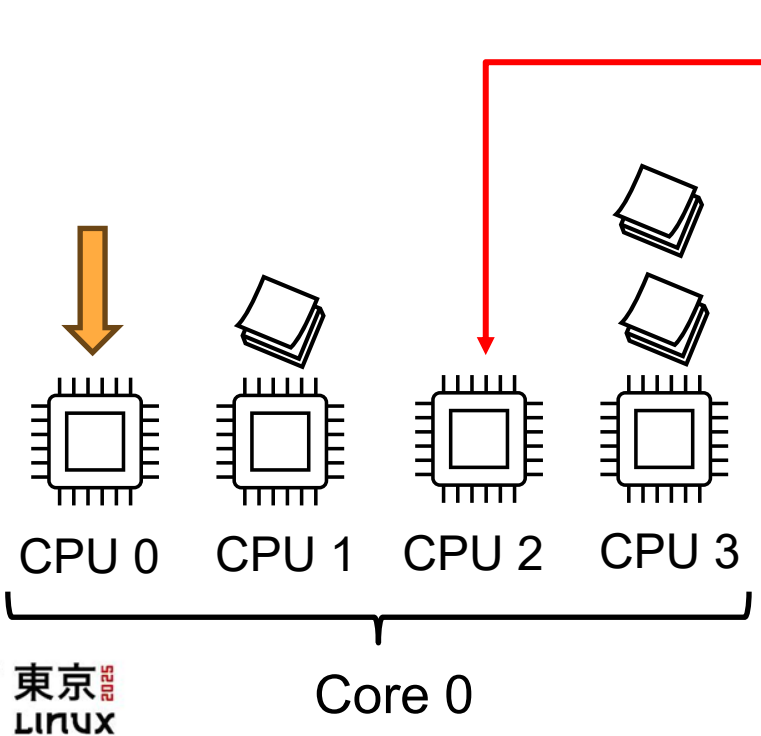
```
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;
        }
#endif

        return true;
}
```

The Behavior of is_core_idle()




```c
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

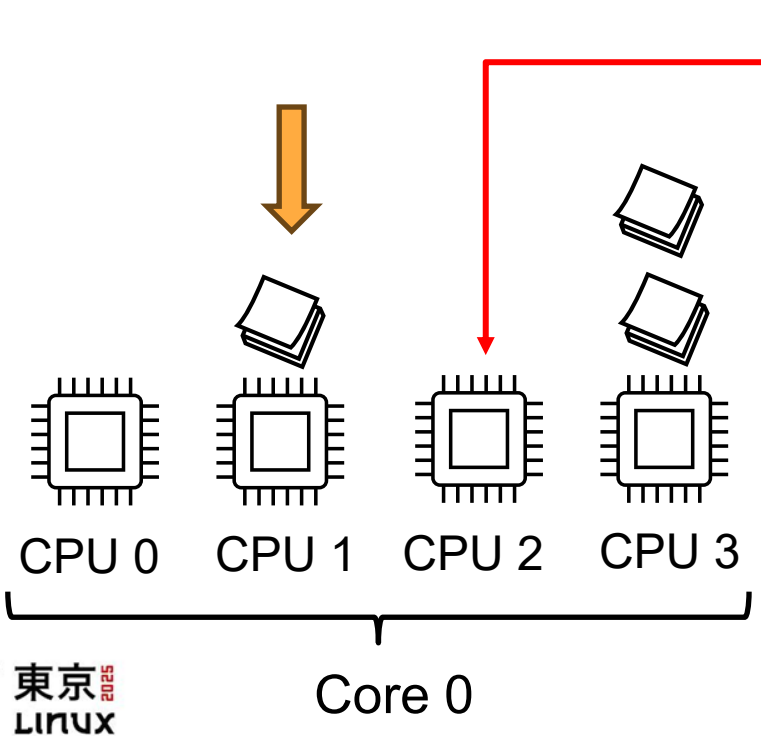
        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;
        }
#endif

        return true;
}
```


CPU Task

The Behavior of is_core_idle()




```c
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

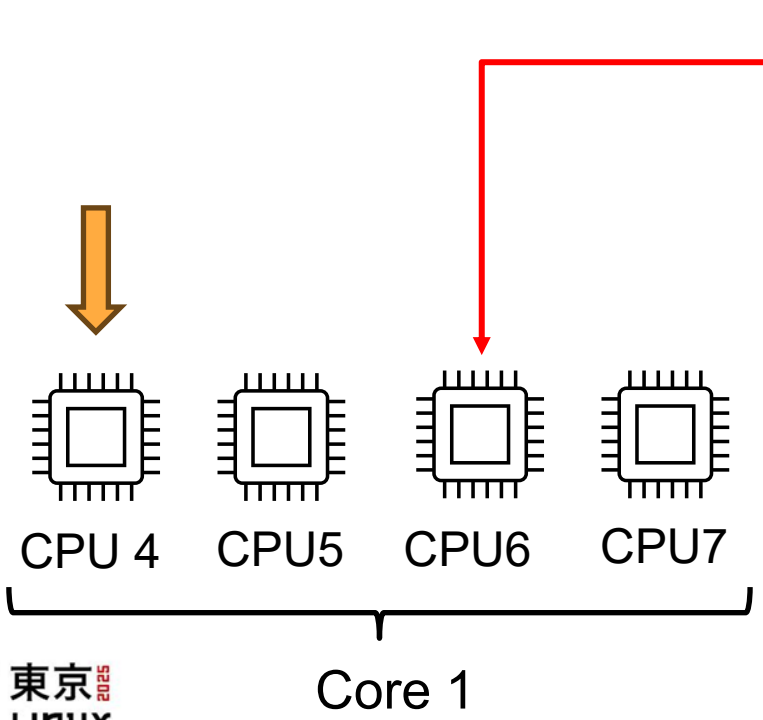
        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;
        }
#endif

        return true;
}
```


CPU Task

The Behavior of is_core_idle()



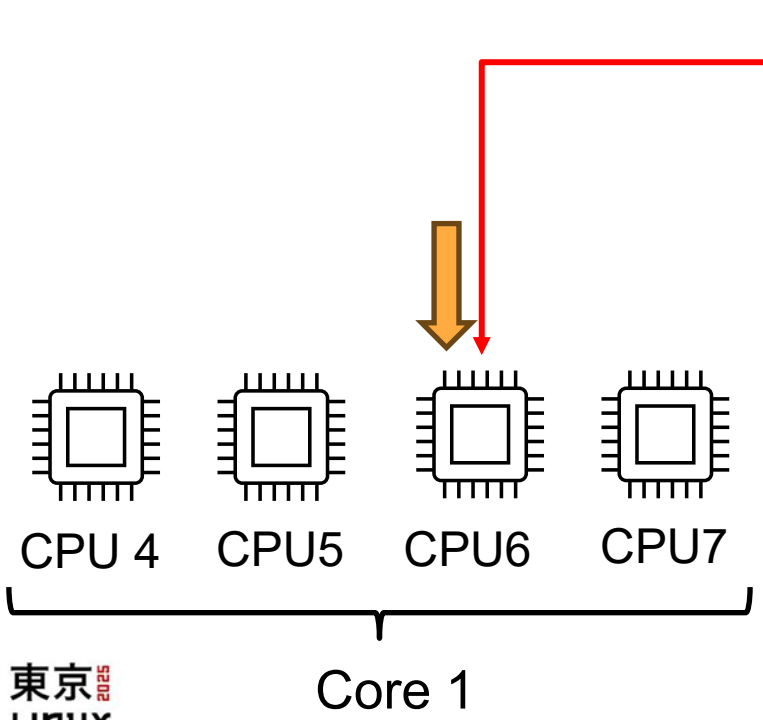

```
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;
        }
#endif

        return true;
}
```

The Behavior of is_core_idle()



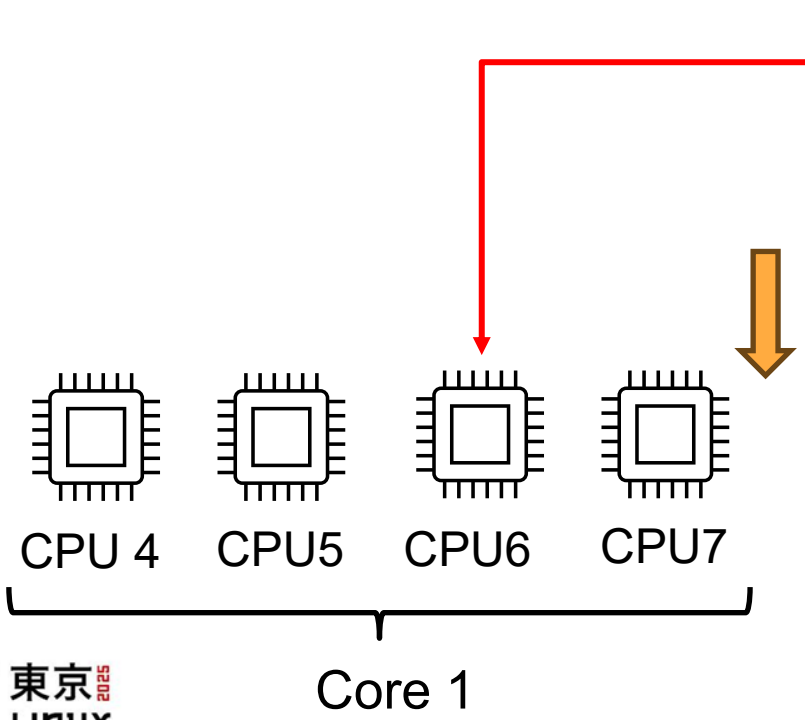

```c
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;
        }
#endif

        return true;
}
```

The Behavior of is_core_idle()




```
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;
        }
#endif

        return true;
}
```

Example: Verification of is_core_idle()

How can we apply verification to this function?

```
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;
        }
#endif

        return true;
}
```

Define Specifications

The specifications of this function

Preconditions

```
requires 0 <= cpu < NR_CPUS;
requires idle_cpu(cpu);
```

Postconditions

```
ensures \result <==> idle_core(cpu);
```

```
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;
        }
#endif

        return true;
}
```

Define Specifications

The specifications of this function

Preconditions

```
requires 0 <= cpu < NR_CPUS;
requires idle_cpu(cpu);
```

Postconditions

```
ensures \result <==> idle_core(cpu);
```

We must define these predicates

```
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;
        }
#endif

        return true;
}
```

Define Specifications

The model of CPU status

```
axiomatic schedule_cpumask {
  predicate idle_cpu(integer cpu);
  logic integer get_core(integer cpu);

  predicate idle_core(integer cpu) =
    \forall integer i; 0 <= i < NR_CPUS ==>
    get_core(i) == get_core(cpu) ==> idle_cpu(i);
}
```

```
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;
        }
#endif

        return true;
}
```

Define Specifications

Th

This predicate defines the idleness of a CPU (*cpu*). We do not give a concrete definition here.

```
axiomatic se    dle_cpumask {
  predicate idle_cpu(integer cpu);
  logic integer get_core(integer cpu);

  predicate idle_core(integer cpu) =
    \forall integer i; 0 <= i < NR_CPUS ==>
    get_core(i) == get_core(cpu) ==> idle_cpu(i);
}
```

```
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;
        }
#endif

        return true;
}
```

Define Specifications

The model of CPU status

```
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;

true;
```

This predicate defines the idleness of a core that a CPU (cpu) belongs to.

```
axiomatic sc
  predicate
  logic inte

  predicate idle_core(integer cpu) =
    \forall integer i; 0 <= i < NR_CPUS ==>
    get_core(i) == get_core(cpu) ==> idle_cpu(i);
}
```

Define Specifications

```
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;
        }
#endif

        return true;
}
```

The model of CPU status

This maps a CPU ID to a core ID.

```
axio
  predicate idl    u(integer cpu);
  logic integer get_core(integer cpu);

  predicate idle_core(integer cpu) =
    \forall integer i; 0 <= i < NR_CPUS ==>
    get_core(i) == get_core(cpu) ==> idle_cpu(i);
}
```

If CPU i belongs to the same core as cpu…

Define Specifications

```
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;
        }
#endif

        return true;
}
```

```
axiomatic schedule_cpumask {
  predicate idle_cpu(integer cpu);
  logic integer get_core(integer cpu);

  predicate idle_core(integer cpu) =
    \forall integer i; 0 <= i < NR_CPUS ==>
    get_core(i) == get_core(cpu) ==> idle_cpu(i);
}
```

Such CPUs must be idle

Applying the Verifier

```
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;
        }
#endif

        return true;
}
```

Recall: a loop requires its invariants

- Iterates over the CPU IDs in the given core.
- Skips the idleness check for the given CPU that is supposed to be idle.
- Otherwise, it checks the idleness. If it is not idle, return early.

Applying the Verifier

```
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;
        }
#endif

        return true;
}
```

The range of the index

```
loop invariant 0 <= sibling <= NR_CPUS;
loop invariant \forall integer j; 0 <= j < sibling ==>
 cpu_smt_mask(cpu)->bits[j] ==> idle_cpu(j);
```

The effect of the loop body

Applying the Verifier

```
static inline bool is_core_idle(int cpu)
{
#ifdef CONFIG_SCHED_SMT
        int sibling;

        for_each_cpu(sibling, cpu_smt_mask(cpu)) {
                if (cpu == sibling)
                        continue;

                if (!idle_cpu(sibling))
                        return false;
        }
#endif

        return true;
}
```

```
loop invariant 0 <= sibling <= NR_CPUS;
loop invariant \forall integer j; 0 <= j < sibling ==>
 cpu_smt_mask(cpu)->bits[j] ==> idle_cpu(j);
```

Some degree of automation is possible?

Outline

- Overview: What is verification?
- Case Study: Applying verification to a small function, is_core_idle()
- Towards practical verification of the Linux kernel

Ideal Verification Workflow for the Kernel

1. Select a function of interest
2. Write a specification
3. Write loop invariants
4. Verify the code

Ideal Verification Workflow for the Kernel

1. Select a function of interest
2. Write a specification
3. Write loop invariants
4. Verify the code

This is not a simple task…

Preparation of Verifiable Code

- What code should be provided to the verifier?
- One can use the source file by adding spec. directly, but:
  - The file contains a lot of irrelevant code.
  - There are many dependencies located in other files.
  - Some dependencies irrelevant to the specifications hinder verification

```
$frama-c -wp kernel/sched/fair.i
[kernel] Parsing kernel/sched/fair.i (no preprocessing)
[kernel] include/uapi/linux/types.h:17: User Error:
  __int128 is currently unsupported by Frama-C.
[kernel] Frama-C aborted: invalid user input.
```

Preparation of Verifiable Code

- Our solution: Automatic preparation of verification code
  - Extracting target functions and dependencies into a minimal but self-contained (compilable) file.
  - Replacing dependencies (types, macros, etc.) with ones suitable for verification based on user-defined instructions.

Preparation of Verifiable Code

- Our solution: Automatic preparation of verification code

- Example: is_core_idle() defined in kernel/sched/fair.c
  - Code Size:
    - fair.c (original code): 13742 lines of code
    - fair.i (macro extended code): 103053 lines of code
    - Our extracted code: 66 lines of code

Preparation of Verifiable Code

- Our solution: Automatic preparation of verification code
- Example
  - Code S
    - fai
    - fair.i (macro e ode): 103053 lines of code
    - Our extracted code: 66 lines of code
      - 9 macros, 4 callees, etc.
      - The dependencies are declared/defined across 10 files

Manual preparation is time-consuming and error-prone.

Ideal Verification Workflow for the Kernel

1. Select a function of interest
2. Write a specification
3. Write loop invariants
4. Verify your code

A tedious task…

Inferring Loop Invariants for the Kernel

- Adapting to the Linux kernel may allow for a practical solution to infer invariants.
- Insight: there are idiomatic usage patterns in the kernel.
  - E.g., for_each_cpu() iterates over CPUs in a mask.
  - More than 400 uses of for_each_cpu() in the kernel.
  - Our current work: automatic invariant inference for specific contexts in the kernel.

```
for_each_cpu(sibling, cpu_smt_mask(cpu)) {
        if (cpu == sibling)
                continue;

        if (!idle_cpu(sibling))
                return false;
}
```

Other Challenges: Lack of Formalized Specifications

- We do not know the formal specifications of most functions in the kernel.
- Maintaining formal specifications is hard and impractical.
- But, once a formal specification is written, the specification is reusable for later updated versions.

Summary

- Deductive Program Verification
  - Pre/Post conditions-based specifications
  - Requirement of manual annotation of loop invariants
  - SMT solvers-based automation
- Towards practical applications:
  - Automating the preparation of verification code and inference of loop invariants **(our current work)**
  - Obtaining good specifications